arXiv:2508.04412v4 [cs.AI] 31 Aug 2026

Beyond Pixels: Exploring DOM Downsampling for LLM-Based Web Agents

Thassilo M. Schiepanski
thassilo@surfly.com

Nicholas Piël
nicholas@surfly.com

Surfly BV

https://github.com/webfuse-com/D2Snap

The advent of large language models (LLMs) has sparked an evolution of autonomous web browsing agents: given a web browsing task and serialised user interface (UI) state, an LLM is expected to suggest input actions that incrementally solve the given task. The central challenge lies in serialising UI state for LLMs. Web agents have increasingly relied on grounded GUI snapshots – screenshots augmented with visual cues – favoured for their modest input token footprint. DOM snapshots – serialised as HTML – represent a compelling alternative that leverages previously demonstrated HTML interpretation capabilities of LLMs. Their excessive input token footprint, however, has precluded reliable deployment with web agents to date.

We propose D2Snap – an algorithm to downsample the DOM, premised on preserving actionability and actionability-discriminating features. We evaluate D2Snap-downsampled DOM snapshots using a snapshot-variant web agent (GPT-4o) on a dataset sampled from Online-Mind2Web. Whilst 42% of raw DOM snapshots exceed the model's context window (128×10^3 tokens), all D2Snap-downsampled DOM snapshots of our reference configuration fit, at a mean context utilisation of 16.5%. Against the 67% success rate of a grounded GUI snapshot baseline, our configuration attains 73% (+5.8%pt; 95% CI −13.6 to +26.0%pt; McNemar, $p = 0.47$), excluding a deficit (one-sided 95%) beyond 11%pt. Image input moreover appears to add little to snapshot utility; grounding text alone attains 62% (−5.8%pt; McNemar, $p = 0.37$).

```
<section class="container" tabindex="3" required="true" type="example">
  <div class="mx-auto" data-topic="products" required="false">
    <h1>Our Pizza</h1>
    <div>
      <div class="shadow-lg">
        <h2>Margherita</h2>
        <p>
          A simple classic: mozzarela, tomatoes and basil.
          An everyday choice!
        </p>
        <button type="button">Add</button>
      </div>
      <div class="shadow-lg">
        <h2>Capricciosa</h2>
        <p>
          A rich taste: mozzarella, ham, mushrooms, artichokes, and olives.
          A true favourite!
        </p>
        <button type="button">Add</button>
      </div>
    </div>
  </div>
</section>
```

```
<section tabindex="3" type="example" class="container" required="true">
  # Our Pizza
  <div class="shadow-lg">
    ## Margherita
    A simple classic: mozzarela tomatoes and basil.
    <button type="button">Add</button>
    ## Capricciosa
    A rich taste: mozzarella ham mushrooms artichokes and olives.
    <button type="button">Add</button>
  </div>
</section>
```

1 Introduction

Web-based user interfaces have traditionally been human-centric and, consequently, developed primarily around visual output and manual input. Artificially intelligent agents that autonomously browse the web on behalf of humans (hereafter referred to as web agents) have been a longstanding subject of research (Amandi and Armentano, 2004; Gur et al., 2024a; He et al., 2024). Such agents are constrained to interact with human-facing user interfaces, as application programming interfaces are often unavailable, incomplete, or restricted (Booking.com, 2024). Conventional AI agents have largely relied on heuristic search over a formal model of the problem domain (Poole and Mackworth, 2017). Because web application UIs are highly complex and diverse, constructing a model adequate for human-centric web browsing is non-trivial. Large language models have recently enabled a new approach to building web agents. In this paradigm, an LLM serves as a reasoning backend: given serialised UI state (hereafter referred to as a snapshot), it is expected to suggest input for advancing a specified web browsing task (Deng et al., 2023; OpenAI, 2024a).

1.1 Problem

Web browser automation frameworks expose perception and actuation interfaces to capture output and send input, respectively (SeleniumHQ, 2026; Microsoft, 2026). A central problem in developing web agents (hereafter used synonymously with LLM-based web agents) is context engineering (Mei et al., 2025; Anthropic, 2025): how best to facilitate an LLM's interpretation of a web page UI to elicit accurate input actions, each specified by a type and a target (e.g., `click` on a specific `button`).

GUI (graphical UI) snapshots, i.e., screenshots, emulate what humans see when viewing a web page. Image input to LLMs is downsampled, which decouples input byte size from input token count (OpenAI, 2025a,b,c). As a result, GUI snapshots of representative real-world web pages fit comfortably within the context window of many frontier LLMs today (OpenAI, 2025d; Anthropic, 2024). LLM vision capabilities still lack pixel precision as image downsampling does not systematically preserve actionable elements. For that reason, state-of-the-art web agents have increasingly been premised on grounded GUI snapshots – screenshots enhanced with visual cues that survive downsampling (Müller and Žunič, 2024; Browser Use, 2025; Yang et al., 2023) (see **Attachment A**). Text-based snapshots – DOM snapshots in particular – present a viable alternative to GUI snapshots. The DOM is a web browser's runtime model of a web page, and HTML is the DOM's canonical serialisation format (WHATWG, 2025b). LLMs have demonstrated the ability to interpret HTML and even navigate UIs indirectly encoded as HTML (Gur et al., 2022). However, DOM snapshots of representative real-world web pages routinely exceed the context window of many LLMs today (Webfuse, 2025).

Enabling DOM snapshots for use with web agents therefore motivates pre-processing techniques, analogous to image pre-processing that LLM providers already implement (OpenAI, 2025a). Such techniques must reduce a snapshot's size without compromising its utility as input to a web agent's LLM backend. Preserving a valid DOM is desirable, ultimately, to allow further processing, such as translation to an accessibility tree. Naïve truncation of the serialised DOM removes actionable elements indiscriminately and breaks HTML syntax.

1.2 Contributions

In this work, we propose *D2Snap*, an algorithm for downsampling the DOM. Intended as a pre-processing technique for use with web agents, it locally consolidates DOM nodes such that the output's utility as a snapshot is largely preserved – trading fidelity for size. Crucially, it remains a valid DOM.

We evaluate D2Snap through a minimal, snapshot-variant agent (*GPT-4o* (OpenAI, 2025d, 2024b) backend) on web browsing tasks derived from *Online-Mind2Web* (Xue et al., 2025b). To avoid subjective judgement, we handcraft a multimodal snapshot dataset along verified solution trajectories (52 snapshots, 18 trajectories), annotated with ground-truth action targets.

Raw DOM snapshots exceed the model's context window (128×10^3 tokens) in 42% of the cases, at a mean model context utilisation of 109%. All snapshots of our reference configuration – low retention of elements, high retention of attributes, moderate retention of text – fit, at a utilisation of 16.5% against 3.0% for a grounded GUI snapshot (Set-of-Mark) (Müller and Žunič, 2024; Yang et al., 2023) baseline. Its 73% success rate is not significantly different from the baseline's 67% rate (+5.8%pt, 95% CI −13.6 to +26.0%pt; McNemar, $p = 0.47$), excluding a deficit (one-sided 95%) beyond 11%pt. Our results support that HTML conveys salience: flattening the DOM to text with inlined actionable elements significantly underperforms our reference (−26.9%pt, $p = 0.002$). Baseline snapshots without images attain 62% (−5.8%pt; $p = 0.37$), indicating that image input adds little to snapshot utility.

2 D2Snap

Downsampling is a technique to reduce data through local, lossy consolidation, such that encoded features are largely preserved (Gonzalez and Woods, 2018). A digital image can be locally consolidated by averaging an n-tile of pixels – the depicted object would remain recognisable (LeCun et al., 1998). We transfer this principle to the DOM: it can analogously be locally consolidated by merging n-subtrees of nodes into a single 'average' node. What constitutes a feature thereby depends on the intended purpose of the downsampled DOM; in our case, features are implied by the downsampled DOM's utility as a snapshot for a web agent's LLM backend. A gradual increase of the downsampling ratio should yield a gradual DOM reduction. Since HTML lexemes have unbounded length, however, monotonic size reduction is a strong aim, and linear size reduction a weak, best-effort aim.

2.1 Algorithm

We propose D2Snap (*Downsample DOM Snapshots*) – an informed algorithm for downsampling the DOM, designed to preserve (A) its utility as a web agent snapshot, and (B) syntactic validity. It figuratively averages redundant nodes of three types: elements, attributes and text (WHATWG, 2025a). To control downsampling per type, we introduce three parameters over the unit interval: the ratios r_e for elements and r_t for text, and the discarding threshold r_a for attributes ($r_e, r_a, r_t \in [0, 1]$, $1 \equiv$ maximal downsampling). **Attachment B** lists examples of D2Snap-downsampled DOM snapshots.

The DOM is a tree structure (WHATWG, 2025a) – size reduction lies in reducing the hierarchy rendered by nested elements. D2Snap averages elements by merging adjacent elements. The parameter r_e governs the removed height fraction to target a DOM height of $\max(1, \lceil h(\text{DOM}) \cdot (1 - r_e) \rceil)$: merge levels are distributed evenly by a Bresenham gate (Bresenham, 1965). A virtual root node is implied, exempt from consolidation, to admit full DOM flattening. A merge describes substituting the deeper candidate element by its own children. This concept of averaging rests on two assumptions: (1) structural segregation decreases with depth, and (2) most attributes are reflexive or forward-transitive (e.g., `id`, or `hidden` (WHATWG, 2025b), respectively). The converse of (2) does not hold – an invisible element can have a visible parent. As downsampling trades fidelity for size, we accept that attributes might become fuzzy or be swallowed[1].

[1]Pruning ineffective (e.g., hidden) subtrees from the DOM beforehand would mitigate attribute semantics drift.

```
switch Class(ELEMENT_NODE):
  case Class.ACTIONABLE:
    break
  case Class.TEXT_FORMATTING:
    TEXT_NODE ← Markdown(ELEMENT_NODE)
    Replace(ELEMENT_NODE, TEXT_NODE)
    break
  default:
    d ← Depth_frozen(ELEMENT_NODE)
    if d > 1 and ⌊d · r_e⌋ > ⌊(d − 1) · r_e⌋:
      Replace(ELEMENT_NODE, ELEMENT_Children)
      break
```

Listing 1: D2Snap consolidates element nodes by substituting an element with its children, distributed evenly through an r_e-ratio Bresenham gate. The root level is exempt, retaining a valid DOM at $r_e = 1$. Absolute measures: actionable elements are preserved as-is, text-formatting elements are translated to Markdown.

Actionable Elements. Elements that semantically afford actionability represent atoms of the DOM-encoded UI – that is, triggers for UI state transitions. Element consolidation idiomatically transfers the concept of downsampling to the DOM. However, as we design D2Snap for use with web agents, we strictly exclude actionable elements from consolidation. For atomicity, actionable elements, such as buttons, should not inflate with content, particularly from parents. Also, actionable elements presumably hold no deep container substructure, so their exclusion only marginally compromises the targeted DOM height.

Text-Formatting Elements. As an absolute size reduction measure, we include translation of HTML-serialised text-formatting elements to token-efficient Markdown equivalents (Gruber, 2004; GitHub, 2019). Most aggressive HTML syntax downsampling ($r_e = r_a = 1, r_t = 0$) then produces a Markdown representation with inlined actionable elements – an input-aware alternative to existing Markdown snapshots (Ango, 2025).

Listing 1 is a pseudocode description of D2Snap's element node sub-procedure.

2.1.1 Attribute Nodes

Attributes are leaf nodes attached to elements, so reducing the DOM in elements implies reducing attributes. There is no evident analogue of local consolidation for attributes, e.g., regarding centrality. To avoid unconditioned survival of heavy attributes, D2Snap discards attributes whose relevance falls below r_a. We are aware of no authoritative such relevance scoring, so we define it as a heuristic represented

through an empirical parameter $\mathcal{A}$ – a lookup table (see **Attachment C**) or scoring function ($\in [0, 1]$). If $\mathcal{A}$ not only scores attributes by relevance, but also calibrates those scores against global attribute frequency, the threshold r_a functions as a latent downsampling ratio – ideally governing removal of the r_a-fraction of attributes from the DOM. **Listing 2** is a pseudocode description of D2Snap's attribute node sub-procedure.

```
if 𝒜(ATTRIBUTE_NODE_Name) < r_a:
  Remove(ATTRIBUTE_NODE)
```

Listing 2: D2Snap removes attributes that score below the threshold r_a. If $\mathcal{A}$ is calibrated against global attribute frequency, r_a functions as a latent downsampling ratio.

2.1.2 Text Nodes

Text nodes embed natural language into the DOM. D2Snap consolidates text by reducing its size at the sentence level (Rai and Borah, 2021). Of the many text-summarisation techniques available, D2Snap uses the *TextRank* algorithm (Mihalcea and Tarau, 2004) for a generalist setup, pruning the floored r_t fraction of sentences with the lowest centrality within the text body, whilst preserving their original order[2]. Single-sentence text nodes, as common for buttons or links, are hence retained. **Listing 3** is a pseudocode description of D2Snap's text node sub-procedure.

```
SENTENCES ← Tokenize(TEXT_NODE_TextContent)
SENTENCES_ranked ← TextRank_Sentence(SENTENCES)
SENTENCES_selected
  ← SENTENCES_ranked[0 : max(⌈(1 − r_t) · |SENTENCES|⌉, 1)]
SENTENCES_re−ordered ← Order_Natural(SENTENCES_selected)
TEXT_NODE_TextContent ← Join(SENTENCES_re−ordered)
```

Listing 3: D2Snap consolidates text nodes on a sentence level: text is tokenised into sentences and ranked by centrality using the TextRank algorithm. The lowest-ranking r_t fraction is discarded; surviving sentences are re-emitted to the text node in their original order. At least one sentence is always retained.

Attachment D displays a full pseudocode description of D2Snap.

3 Evaluation

Our experimental setup to evaluate D2Snap isolates the snapshot representation as the independent variable, and its utility as LLM input as the dependent variable. Our principal research questions are:

- **RQ 1.** Do D2Snap-downsampled DOM snapshots fit comfortably within a frontier LLM's context window? We operationalise a comfortable fit as mean model context utilisation ($\text{CU}_{\%}$) of at most 20%; leaving headroom for two-snapshot history (3× 20%), additional history context (2× 10%) and a system prompt (20%).
- **RQ 2.** Do D2Snap-downsampled DOM snapshots allow a frontier LLM to suggest input action targets that advance real-world web browsing tasks?

3.1 Evaluation Dataset

To this end, we construct a dedicated snapshot dataset, sourced from Online-Mind2Web (Xue et al., 2025b). We randomly sample 6 easy, 6 medium, and 6 hard web browsing tasks.

In a first pass, a human (non-author) is asked to solve each sampled task, for which we collect a solution trajectory of snapshots[3]. Each snapshot is a triple of independent representations: a DOM snapshot, a GUI snapshot, and a grounded GUI snapshot, emulating the Set-of-Mark (Yang et al., 2023) approach of *Browser Use* (Müller and Žunič, 2024). We verify all trajectories via *WebJudge* (NLP, 2025b); all 18 trajectories passed verification. The resulting dataset contains a total of 52 tuples of tasks and snapshot-triples – each snapshot along a trajectory is treated as an independent sub-task.

In a second pass, two software engineers (non-authors) independently annotate every snapshot triple, identifying all coherent sets of input action targets. A coherent action target set is a minimal set of UI elements that requires input (e.g., clicking or typing) to advance a related web browsing task, i.e., transition between adjacent, encoded UI states. A snapshot may admit multiple coherent target sets (e.g., a redundant link in the `header` and the `footer`). An annotation takes the form of bounding-box coordinates for a GUI snapshot, supplemented with a numerical identifier (if available) for a grounded GUI snapshot, and a unique CSS selector for a DOM snapshot. For assessing snapshot utility, we do not verify action target arguments, nor order within coherent action target sets. Annotations (A, B) yield an inter-annotator agreement of F1 = 0.89[4]. Given high agreement, we take either annotator's targets to

[2] TextRank on sentences is meaningless for few sentences. For short texts, it can be substituted, e.g., by truncation.

[3] Snapshots are taken before each input (Xue et al., 2025a).

[4] Pairwise F1 between both annotators' action target sets ($|A| = 471$, $|B| = 499$, $|A \cup B| = 538$, $|A \cap B| = 432$). Two annotated action targets match if: (1) they share the same snapshot, (2) they share the same representation, and (3), depending on the representation, their bounding boxes overlap

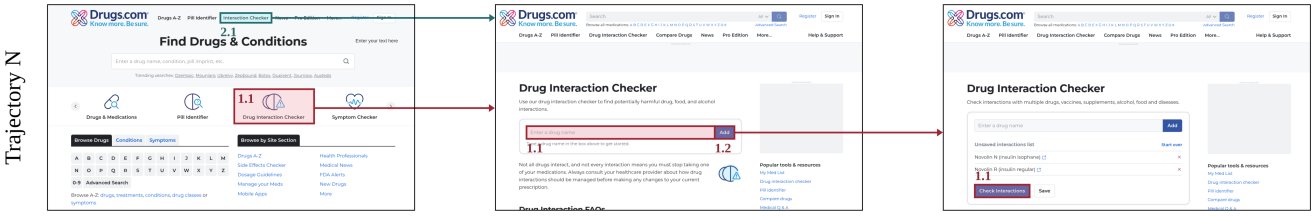


Figure 1: Annotated solution trajectory of UI states encoded as GUI snapshots from our dataset on *drugs.com* for the task: *"Check the interaction between Novolin N and Novolin R"*. Coloured bounding boxes represent annotated action targets. A group of equally coloured boxes forms a coherent target set. A web agent must interact with all elements of a single coherent target set to transition across the UI state trajectory. Our dataset contains full-page GUI snapshots.

be valid and build the evaluation reference from the union[5], i.e., ground-truth action targets against which LLM-suggested targets can be matched. **Figure 1** shows an example annotation for a trajectory of GUI snapshots. **Attachment E** excerpts a dataset reference annotation.

3.2 Evaluation Framework

The evaluation of snapshot representations is designated to deploy a web agent across each tuple from the dataset, once per representation. The web agent LLM backend's response is counted as a success if the suggested action targets form a superset of any coherent target set in the reference[6][7]. At the aggregate level, the evaluation success rate is computed over all 52 task-snapshot tuples per representation.

3.3 Experimental Setup

We deploy a minimal web agent – merely a model with a snapshot-variant system prompt – to attribute performance differences to snapshot representations. As the backend, we choose GPT-4o (`gpt-4o-2024-11-20` (OpenAI, 2025d)), a multimodal, general-purpose model, with a context window of 128×10^3 tokens ($\equiv$ 100% $\mathrm{CU}_{\%}$)[8]. Rather than establishing heuristic attribute scoring manually, we elicit it directly from the model, independently of the dataset, which presupposes the model's capacity to interpret HTML semantics – the enabling assumption of LLM-based DOM interpretation. Additionally, we write a system prompt template instructing the model to suggest all action targets that advance a given task on a web page encoded by a given snapshot, and substitutions specific to each snapshot representation. With DOM snapshots, for instance, element targeting is described by means of CSS selectors. **Attachment C** records our experimental attribute scoring lookup table, including eliciting model prompts. **Attachment F** records the exact system prompt templates used in our experiments.

Snapshot Representations. As the baseline snapshot technique, we adopt grounded GUI snapshots, as implemented by *Browser Use* (Browser Use, 2025) following Set-of-Mark (Yang et al., 2023) (A). Our evaluation compares the baseline against its grounding text alone, i.e., without the image (B), raw GUI snapshots (C), and raw DOM snapshots (D). It covers a range of D2Snap-downsampled DOM snapshots compiled from raw DOM snapshots from the dataset (E, F), including a linearised Markdown representation with inlined actionable HTML (G). **Attachment A** shows an example of a GUI snapshot and its equivalent grounded GUI snapshot. The evaluated snapshot representations are referred to by class notation:

by IoU $\geq$ 0.75, their numeric IDs are equal, or their CSS selectors resolve to the same element.

[5]Two matched bounding boxes are united as the minimal rectangle enclosing both.

[6]Elements are compared with slight tolerance: for GUI snapshots, we inflate referenced bounding boxes by 10 pixels to each side; for DOM-based snapshots, we additionally accept a referenced node's parent. The latter accommodates input event bubbling behaviour (Stchur and Steiner, 2021). Although expressed in different units, both tolerances absorb representation-specific ambiguity: pixel-level annotation jitter (GUI), and handler uncertainty (DOM).

[7]The agent could trivially satisfy the success criterion by excessively suggesting action targets. We mitigate such degenerate suggestions by rejecting target sets exceeding 3$\times$ the size of the largest coherent reference set. The buffer accounts for additional, non-interfering action targets (e.g., an info-box). Across our experiments, the ceiling was reached at most twice per snapshot representation.

[8]For reproducibility, agent response temperature is fixed at 0. Results per snapshot representation are averaged across three runs; booleans are averaged by majority.

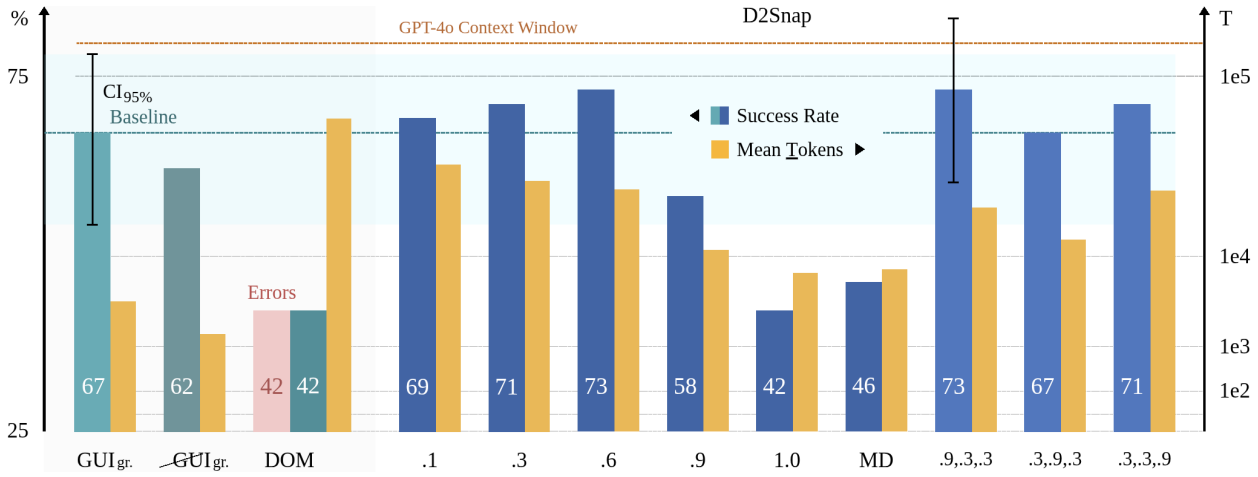


Figure 2: Agent success rates and mean token sizes across snapshot representations plotted as a bar chart. Grounded GUI snapshots ($\text{GUI}_{\text{gr.}}$) represent the baseline (67%, 95% Wilson CI: 54% to 78%). 42% of raw DOM snapshots exceeded the model's context window (GPT-4o, 128×10^3 tokens). Every D2Snap-downsampled snapshot fits. Success with D2Snap-based representations is not significantly different from the baseline (e.g., $\text{D2Snap}_{.9,.3,.6}$, 73%; +5.8%pt, 95% CI −13.6 to +26.0%pt; McNemar, $p = 0.47$). Grounding text alone ($\text{GUI}_{\text{grounded}}$) reaches 62% (McNemar, $p = 0.37$).

A.	$\textbf{GUI}_{\text{grounded}}$	**Baseline**
B.	$\text{GUI}_{\text{grounded}}$	$\text{GUI}_{\text{grounded}} \setminus \text{image}$
C.	**GUI**	raw screenshots
D.	$\textbf{DOM}^{9}$	raw serialised DOM (HTML)
E.	$\textbf{D2Snap}_{r_e,r_a,r_t}$	$r_e, r_a, r_t \in [0,1]^{10}$
F.	$\textbf{D2Snap}_{r}$	$r = r_e = r_a = r_t$
G.	$\textbf{D2Snap}_{\text{MD}}$	$r_e = r_a = 1, r_t = 0$

3.4 Experimental Results

Raw DOM snapshots (**DOM**) exceed the model's context window in 42% of the cases, at a mean $\text{CU}_{\%}$ of 109%. Even at moderate downsampling ratios, every D2Snap-downsampled snapshot fits, and 13 of 15 assessed configurations have at most 20% $\text{CU}_{\%}$ (**RQ 1**)[11]. At a $\text{CU}_{\%}$ of 16.5%, our reference configuration $\textbf{D2Snap}_{.9,.3,.6}$ achieves a success rate of 73%, which is not significantly different from the baseline ($\textbf{GUI}_{\text{grounded}}$) rate of 67% (+5.8%pt, 95% CI −13.6 to +26.0%pt; McNemar, $p = 0.47$) – excluding a deficit (one-sided 95%) beyond 11%pt (**RQ 2**). Text-linearised representations ($\textbf{D2Snap}_{1.0}$, $\textbf{D2Snap}_{\text{MD}}$) attain the lowest success rates among D2Snap configurations, underperforming our reference configuration significantly (42%, −30.8%pt, $p < 0.001$; 46%, −26.9%pt, $p = 0.002$). **Table 1** compares all evaluated snapshot representations on agent success, snapshot size, and LLM backend latency. **Figure 2** visualises success rate and token count among key representations. **Figure 3** plots token counts of every output of our reference D2Snap configuration.

At a success rate of 2%, LLM vision capabilities evidently lack pixel-precision. Grounding is therefore necessary to make GUI snapshots effective

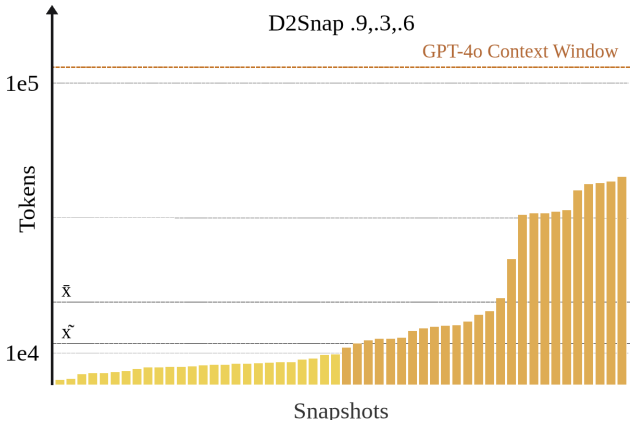


Figure 3: Ordered token sizes for each D2Snap-downsampled DOM from the dataset, produced by $\textbf{D2Snap}_{.9,.3,.6}$. All snapshots fit within the model's context window (GPT-4o, 1.28×10^5 tokens), at a mean utilisation of 16.5% and a maximum of 54%. 50% are below 10^4 tokens.

[11] We estimate LLM input tokens from serialised byte size. We map targets between original DOM and downsampled counterparts by assigning unique data attributes to the DOM. These are excluded from size measurements, as they stem from an opinionated mapping approach – a source map would not affect snapshots directly. To not introduce bias, these attributes are applied to every element and values are incremental integers.

	Success (%)		$\bar{x}$ Size ($\times 10^3$)					$\bar{x}$ Latency	
	Rate	Wilson $CI_{95\%}$	Bytes (B)	Tokens (T)	$P_{5,95}$ (T)	$CU_{\%}$	Δ_{Raw}	$\hat{\Delta}$	SEM
GUI$_{grounded}$ (Baseline)	**67.3**	[53.8, 78.5]	2,378.22	3.78	0.65, 11.38	**3.0**	–	**3.42**	0.27
~~**GUI**~~$_{grounded}$	**61.5**	[48.0, 73.5]	5.84	1.46	0.36, 3.84	**1.1**	–	1.00	0.07
GUI	1.9	[0.3, 10.1]	2,349.33	2.29	0.17, 7.63	1.8	–	**3.40**	0.24
DOM	**42.3**	[29.9, 55.8]	559.10	139.78	13.39, 498.39	**109.2**	0.93	3.05	0.28
D2Snap$_{.1}$	69.2	[55.7, 80.1]	123.00	30.73	5.21, 93.61	24.0	0.28	2.31	0.22
D2Snap$_{.3}$	71.2	[57.7, 81.7]	104.38	26.07	4.69, 84.02	20.4	0.24	2.50	0.25
D2Snap$_{.6}$	**73.1**	[59.7, 83.2]	93.37	23.32	3.93, 76.74	18.2	0.20	2.46	0.24
D2Snap$_{.9}$	57.7	[44.2, 70.1]	42.55	10.62	1.73, 30.47	8.3	0.11	1.72	0.17
D2Snap$_{1.0}$	**42.3**	[29.9, 55.8]	18.77	4.69	1.11, 10.71	**3.7**	**0.06**	**1.38**	0.11
D2Snap$_{MD}$	**46.2**	[33.3, 59.5]	20.96	5.24	1.13, 11.42	4.1	**0.06**	1.49	0.15
D2Snap$_{.9,.3,.3}$	**73.1**	[59.7, 83.2]	85.83	21.46	3.71, 67.00	16.8	0.19	2.14	0.19
D2Snap$_{.3,.9,.3}$	67.3	[53.8, 78.5]	48.88	12.22	1.91, 32.79	9.5	0.13	1.79	0.15
D2Snap$_{.3,.3,.9}$	71.2	[57.7, 81.7]	102.38	25.59	4.52, 83.57	20.0	0.23	2.18	0.19
D2Snap$_{.3,.6,.9}$	69.2	[55.7, 80.1]	100.45	25.11	4.37, 83.32	19.6	0.22	2.42	0.24
D2Snap$_{.3,.9,.6}$	65.4	[51.8, 76.8]	47.55	11.89	1.89, 32.24	9.3	0.12	1.76	0.16
D2Snap$_{.6,.3,.9}$	**73.1**	[59.7, 83.2]	94.56	23.64	4.14, 76.77	18.5	0.21	2.08	0.20
D2Snap$_{.6,.9,.3}$	55.8	[42.3, 68.4]	47.03	11.76	1.83, 32.12	9.2	0.12	1.79	0.17
D2Snap$_{.9,.3,.6}$	**73.1**	[59.7, 83.2]	84.50	21.13	3.69, 66.73	**16.5**	0.19	**2.42**	0.23
D2Snap$_{.9,.6,.3}$	67.3	[53.8, 78.5]	84.05	21.01	3.51, 66.81	16.4	0.18	2.22	0.21

Table 1: Agent evaluation results across snapshot representations over $n = 52$ annotated snapshots, averaged across three runs at temperature 0: agent task success rates with marginal 95% Wilson intervals, which are per representation, not a basis for pairwise comparison. Mean byte and token sizes with token percentiles (5th, 95th; wide, as individual web page sizes vary). Mean model context utilisation ($CU_{\%}$, GPT-4o, 128×10^3 tokens). Mean snapshot size ratio against the mean raw DOM snapshot from the dataset (Δ_{Raw}). Mean latency normalised to the lowest recorded value (~~GUI~~$_{grounded}$) with 1 SEM. Grounded GUI snapshots ($GUI_{grounded}$) represent the baseline. 42% of raw DOM snapshots exceeded the context window. Success with D2Snap-based representations is not significantly different from the baseline (e.g., $D2Snap_{.9,.3,.6}$, +5.8%pt, McNemar, $p = 0.47$), nor with grounding text alone (~~GUI~~$_{grounded}$, McNemar, $p = 0.37$).

(**GUI**$_{grounded}$, 67%). However, we detect no significant difference when using the grounding text without screenshots as standalone snapshots (~~**GUI**~~$_{grounded}$); the agent attains a success rate of 62% (−5.8%pt, 95% CI −18.0 to +7.1%pt; McNemar, $p = 0.37$), which suggests that the text, rather than the image input, carries most utility-inherent features.

Ablation Study. We infer the impact on snapshot utility of each D2Snap parameter by raising a single parameter to 0.9 (high) from an otherwise uniform 0.3 (low). Against the uniform **D2Snap**$_{.3}$ reference (71%, $\Delta_{Raw} = 0.24$), this directionally yields: (A) r_e – low retention of elements (hierarchy) – reduces size by 18% at 73%, so structural segregation can be traded almost freely; (B) r_a – low retention of attributes – reduces size by 53%, the largest saving of any parameter, but drops to 67%, indicating that attributes add most to utility; and (C) r_t – low retention of text – neither reduces size (1.9%) nor changes success (71%) noticeably. The embedded text consolidation technique (we deployed TextRank) narrows down to a semantic-to-verbosity optimisation problem. Notably, retaining HTML at low fidelity (**D2Snap**$_{.6}$, 73%) outperforms full removal of HTML (**D2Snap**$_{1.0}$, 42%; +30.8%pt, McNemar, $p < 0.001$) – supporting the use of DOM snapshots over abstract text-based snapshot representations. Retaining text at full length (**D2Snap**$_{MD}$, 46%) does not measurably change utility over discarding it (**D2Snap**$_{1.0}$; +3.8%pt, $p = 0.32$), suggesting that natural language text verbosity contributes little to a DOM snapshot's utility.

Range Study. D2Snap's output ranges from the original HTML (**D2Snap**$_0$) to fully flattened content – virtually Markdown with inlined actionable elements (**D2Snap**$_{1.0}$). We record the uniform configurations $r = r_e = r_a = r_t \in \{0.1, \ldots, 1.0\}$. Size reduction is strictly monotonic across this range (Spearman $\rho = -1.000$), satisfying our strong aim. It further approximates a constant reduction rate: a linear regression of the mean size ratio against r over the operating range fits $\hat{\Delta}_{Raw} = 0.311 - 0.226\,r$ (Pearson correlation −0.965, $R^2 = 0.93$), satisfying our weak aim. The drop from 0.7 to 0.8 stands out: it is due to the `class` attribute (WHATWG, 2025b), which our experimental attribute scoring places at 0.77 and which is therefore discarded once r_a exceeds that score. Aligned with prior expectation, this is a high-frequency attribute. Restricting the regression to

	$\bar{x}$ Size		
	Tokens ($\times 10^3$)	Δ_{Raw}	Δ
raw	111.74	1	–
D2Snap$_{.1}$	30.73	0.275	−0.725
D2Snap$_{.2}$	28.26	0.253	−0.022
D2Snap$_{.3}$	26.07	0.238	−0.015
D2Snap$_{.4}$	25.29	0.226	−0.012
D2Snap$_{.5}$	23.58	0.208	−0.018
D2Snap$_{.6}$	23.32	0.202	−0.006
D2Snap$_{.7}$	21.37	0.183	−0.019
D2Snap$_{.8}$	11.50	0.115	−0.068
D2Snap$_{.9}$	10.62	0.109	−0.006
D2Snap$_{1.0}$	4.69	0.057	−0.052

Table 2: Mean token counts and size ratio against the mean raw DOM snapshot from the dataset (Δ_{Raw}) over uniform D2Snap parameter configurations $r = r_e = r_a = r_t \in \{0.1, \ldots, 1.0\}$, supplemented with absolute change (Δ). The progression is strictly monotonic (Spearman $\rho = -1.000$). Over the operating range r, it fits $\hat{\Delta}_{\text{Raw}} = 0.311 - 0.226\,r$ (Pearson correlation -0.965, $R^2 = 0.93$). The drop from 0.7 to 0.8 is due to the elimination of the high-frequency `class` attribute.

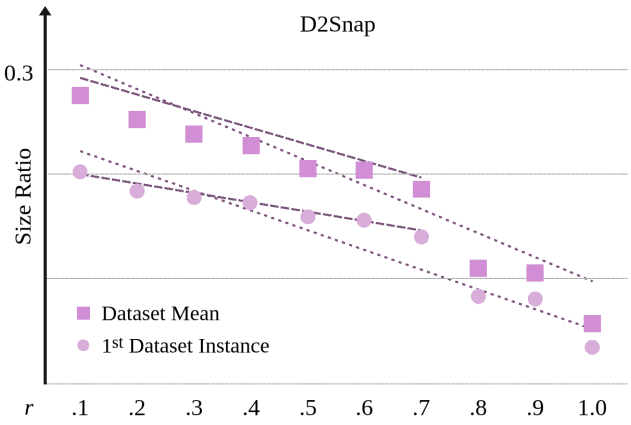


Figure 4: Mean size ratio against the mean raw DOM snapshot from the dataset (Δ_{Raw}) over uniform D2Snap parameter configurations $r = r_e = r_a = r_t \in \{0.1, \ldots, 1.0\}$. The progression is strictly monotonic (Spearman $\rho = -1.000$). Over the operating range r, it fits $\hat{\Delta}_{\text{Raw}} = 0.311 - 0.226\,r$ (Pearson correlation -0.965, $R^2 = 0.93$). The drop from 0.7 to 0.8 is due to the elimination of the high-frequency `class` attribute. The size progression on the first snapshot from the dataset resembles the trend.

$r \in \{0.1, \ldots, 0.7\}$, below that threshold, yields a closer fit of $\hat{\Delta}_{\text{Raw}} = 0.285 - 0.146\,r$ (Pearson correlation -0.993, $R^2 = 0.99$). **Table 2** reports mean snapshot token counts and size ratios across the recorded range; **Figure 4** plots the size ratios against the regressions.

4 Related Work

Context Engineering. Input token efficiency has been approached with increasing web agent specificity: LLMLingua (Jiang et al., 2023, 2024; Pan et al., 2024) discards low-saliency tokens by perplexity in continuous natural language prose – a noteworthy alternative to our TextRank component of D2Snap. Set-of-Mark prompting augments a GUI snapshot with visual cues (Yang et al., 2023), which has influenced state-of-the-art production agents such as *Browser Use* (Browser Use, 2025). Token-extraction and -stripping approaches reduce HTML directly, yet apply uniform transformations that disregard differential DOM node semantics (Chang et al., 2006; Gupta et al., 2003). Closest to our work, structured-text representations serialise the page directly: most obviously, the accessibility tree (Boxhall et al., 2016) derived from the DOM (Zhou et al., 2024). Markdown constitutes an orthogonal representation (Ango, 2025). Such representations typically follow element extraction – surfacing a designated subset of elements for translation. Element extraction has built on above-outlined transformations, but also on LLM inference, such as relevance ranking or predictors conditioned on prior actions (Gur et al., 2024b; Deng et al., 2023) – at the cost of added latency. In contrast, D2Snap downsamples the full DOM under an explicit fidelity-size trade-off, preserving features that constitute or discriminate action targets across the entire representation at a controllable ratio, and emitting a valid DOM.

Web Agents. A growing body of work has proposed architectures for generalist, LLM-based web agents (Kim et al., 2023; Gur et al., 2024a; Zhou et al., 2024; He et al., 2024; Zheng et al., 2023; Müller and Žunič, 2024; OpenAI, 2025e) – based on DOM, GUI or hybrid snapshots. A parallel line of work has focused on web agent evaluation: Mind2Web (Deng et al., 2023) introduced a static dataset of web browsing tasks to assess web agent success. Online-Mind2Web (Xue et al., 2025b) is a smaller-scale derivative targeting execution on the live web; WebVoyager (He et al., 2024) likewise targets the live web. These datasets are designed to capture actual web browsing journeys rather than final results (NLP, 2025a). The evaluation of snapshot utility in isolation, however, has received comparatively little attention – generalist web agent evaluation results merely allow indirect assessment of snapshot representations. Our work closes this gap, presenting a static dataset to assess different snapshot representations. Our related experimental framework

is designed to assess the impact of a specific snapshot representation independently of individual web agent logic. It does, moreover, not require human or AI judgement (e.g., WebJudge (NLP, 2025b)[12]), as it defines objective rules to determine success.

5 Conclusion

DOM snapshots – if not for their excessive LLM input footprint – represent a compelling alternative to GUI snapshots. In this paper, we introduced DOM downsampling: trading DOM fidelity for size, whilst preserving a valid DOM.

5.1 Contributions

We propose D2Snap – an algorithm to downsample the DOM, premised on preserving actionability and actionability-discriminating features, intended as a size-reducing pre-processing technique for DOM snapshots used with web agents. On D2Snap-downsampled DOM snapshots, a minimal, snapshot-variant web agent (GPT-4o, context size: 128×10^3 tokens) achieves a web browsing task success rate that is not significantly different from that of a grounded GUI snapshot baseline (+5.8%pt, 95% CI −13.6 to +26.0%pt), excluding a deficit (one-sided 95%) beyond 11%pt, at a mean model context utilisation of 16.5%. Every D2Snap-downsampled snapshot fits within the context window; 42% of raw DOM snapshots exceed it. We further detect no significant difference in success between grounded GUI snapshots and the grounding text alone (−5.8%pt, 95% CI −18.0 to +7.1%pt), which suggests that image input contributes little to snapshot utility. Ablation on success indicates that retention of attributes contributes most to snapshot utility. In turn, low retention of element hierarchy is sufficient (73%), yet full removal of HTML is not (46%). We show that D2Snap downsamples strictly monotonically and approximately linearly across its operating range (linear regression: slope −0.226, $R^2 = 0.93$).

The data and code repository associated with this paper is publicly available at github.com/webfuse-com/D2Snap, mirrored at github.com/t-ski/D2Snap.

5.2 Limitations

Evaluation. Our evaluation dataset has a modest size ($n = 52$), which limits the precision of our estimates. Our reference configuration was selected among the results. It ties with three further configurations at the highest observed success rate, and among those exhibits the lowest context utilisation – a criterion independent of success. Our evaluation relies exclusively on GPT-4o, so results should be read as indicative of snapshot utility for few real-world-representative web pages and a highly capable model. Elicited once, our attribute scoring is best regarded as a model-informed prior.

Integration. Web browsers strictly constrain DOM-access to cross-origin documents embedded via the `iframe` element (WHATWG, 2025a,b). Whilst GUI snapshots are able to capture what is visually rendered within embedded documents' viewports (Microsoft, 2025), DOM snapshots are not. The same limitation holds for graphics-based applications via the `canvas` element (WHATWG, 2025b).

5.3 Future Work

Evaluation. Automated, systematic exploration of the downsampling parameter space – across a broader range of frontier LLMs and web applications – would identify optimal configurations and establish generalisability of the approach. Moreover, we elicited attribute scoring heuristics from a single model under a generic eliciting prompt; future work could investigate alternative elicitation strategies.

Integration. Since D2Snap preserves a valid DOM, it can serve as an intermediate size control measure within a snapshot pipeline. In a subsequent processing step, for instance, the downsampled DOM could be translated to an accessibility tree. In a preceding step, certain subtrees could be pruned, e.g., due to invisibility or redundancy.

Beyond Web Agents. The applicability of DOM downsampling is not limited to the web agent domain. Other domains that might benefit from the concept include accessibility, content summarisation, and UI testing – all of which rely on the DOM.

[12] We utilised WebJudge once to validate task solution UI state trajectories in our dataset. It is not used in our repeatable experimental framework.

References

Analia Amandi and Marcelo Armentano. Connecting web applications with interface agents. *International Journal of Web Engineering and Technology*, 1(4): 454–470, 2004. doi: 10.1504/IJWET.2004.006270.

Steph Ango. github.com/defuddle, 2025.

Anthropic. Claude 3.5 Sonnet. https://www.anthropic.com/news/claude-3-5-sonnet, June 2024. Accessed: 2025-06-02.

Anthropic. Effective context engineering for ai agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, 2025. Accessed: 2025-06-02.

Booking.com. Demand API. https://developers.booking.com/demand, 2024. Accessed: 2025-06-02.

Alice Boxhall, Dave Gash, and Meggin Kearney. The accessibility tree. *web.dev*, 2016. URL https://web.dev/articles/the-accessibility-tree.

Jack E. Bresenham. Algorithm for computer control of a digital plotter. *IBM Systems Journal*, 4(1):25–30, 1965. doi: 10.1147/sj.41.0025.

Browser Use. github.com/browser-use: browser_use/dom/dom_tree/index.js. https://github.com/browser-use/browser-use/blob/6458bbb05e9e65c96d4269021603ce0b16857372/browser_use/dom/dom_tree/index.js, 2025. Accessed: 2025-06-02.

Chia-Hui Chang, M. Kayed, M.R. Girgis, and K.F. Shaalan. A survey of web information extraction systems. *IEEE Transactions on Knowledge and Data Engineering*, 18(10):1411–1428, 2006. doi: 10.1109/TKDE.2006.152.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 28091–28114. Curran Associates, Inc., 2023.

GitHub. Github flavored markdown spec. https://github.github.com/gfm/, 2019. Accessed: 2025-06-02.

Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 4th edition, 2018. ISBN 978-0-13-335672-4.

John Gruber. Daring Fireball: Markdown. https://daringfireball.net/projects/markdown/, 2004. Accessed: 2025-06-02.

Suhit Gupta, Gail Kaiser, David Neistadt, and Peter Grimm. DOM-based content extraction of html documents. In *Proceedings of the 12th International Conference on World Wide Web*, WWW '03, page 207–214, New York, NY, USA, 2003. Association for Computing Machinery. ISBN 1581136803. doi: 10.1145/775152.775182.

Izzeddin Gur, Ofir Nachum, Yingjie Miao, Mustafa Safdari, Austin Huang, Aakanksha Chowdhery, Sharan Narang, Noah Fiedel, and Aleksandra Faust. Understanding HTML with large language models. *ArXiv*, abs/2210.03945, 2022.

Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world webagent with planning, long context understanding, and program synthesis, 2024a.

Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world webagent with planning, long context understanding, and program synthesis. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Learning Representations*, volume 2024, pages 52690–52717, 2024b. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/e91bf7dfba0477554994c6d64833e9d8-Paper-Conference.pdf.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. WebVoyager: Building an end-to-end web agent with large multimodal models, 2024.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.825.

Huiqiang Jiang, Qianhui Wu, , Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

Geunwoo Kim, Pierre Baldi, and Stephen Marcus McAleer. Language models can solve computer tasks. *ArXiv*, abs/2303.17491, 2023.

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11): 2278–2324, 1998. doi: 10.1109/5.726791.

Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A survey of context engineering for large language models, 2025.

Microsoft. Screenshots. https://playwright.dev/docs/screenshots, 2025. Accessed: 2025-06-02.

Microsoft. Playwright. https://playwright.dev/, 2026. Accessed: 2025-05-28.

Rada Mihalcea and Paul Tarau. TextRank: Bringing order into text. In Dekang Lin and Dekai Wu, editors, *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 404–411, Barcelona, Spain, July 2004. Association for

Computational Linguistics.

Magnus Müller and Gregor Žunič. Browser Use: Enable AI to control your browser, 2024.

OSU NLP. Online Mind2Web Leaderboard. https://huggingface.co/spaces/osunlp/Online_Mind2Web_Leaderboard, 2025a. Accessed: 2025-06-02.

OSU NLP. WebJudge-7b. https://huggingface.co/osunlp/WebJudge-7B, 2025b. Accessed: 2025-06-02.

OpenAI. Introducing structured outputs in the api. https://openai.com/index/introducing-structured-outputs-in-the-api, 2024a. Accessed: 2025-06-02.

OpenAI. Gpt-4o system card. https://cdn.openai.com/gpt-4o-system-card.pdf, 2024b. Accessed: 2025-06-02.

OpenAI. Images and vision – limits. https://platform.openai.com/docs/guides/images-vision#limitations, July 2025a. Accessed: 2025-06-02.

OpenAI. Images and vision – calculating costs. https://platform.openai.com/docs/guides/images-vision?api-mode=responses#calculating_costs, July 2025b. Accessed: 2025-05-27.

OpenAI. Counting tokens. https://developers.openai.com/api/docs/guides/token-counting, 2025c. Accessed: 2025-06-02.

OpenAI. Gpt-4o. https://developers.openai.com/api/docs/models/gpt-4o, 2025d. Accessed: 2025-06-02.

OpenAI. Introducing operator. https://openai.com/index/introducing-operator, 2025e. Accessed: 2025-06-02.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Ruhle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics ACL 2024*, pages 963–981, Bangkok, Thailand and virtual meeting, August 2024. Association for Computational Linguistics.

David L. Poole and Alan K. Mackworth. *Artificial Intelligence: Foundations of Computational Agents*. Cambridge University Press, Cambridge, UK, 2 edition, 2017. ISBN 9781108173773. doi: 10.1017/9781108234184.

Abigail Rai and Samarjeet Borah. Study of various methods for tokenization. In Jyotsna K. Mandal, Somnath Mukhopadhyay, and Alak Roy, editors, *Applications of Internet of Things*, pages 193–200, Singapore, 2021. Springer Singapore. ISBN 978-981-15-6198-6.

SeleniumHQ. Selenium. https://www.selenium.dev/, 2026. Browser automation framework. Accessed: 2026-07-08.

Stephen Stchur and Thomas Steiner. Javascript eventing deep dive. *web.dev*, 2021. URL https://web.dev/articles/eventing-deepdive.

Webfuse. github.com/d2snap: dataset/dom/foxsports-0.html/foxsports-0.html. https://github.com/webfuse-com/D2Snap/blob/main/dataset/dom/foxsports-0.html, 2025. Accessed: 2025-06-02.

WHATWG. DOM living standard. https://dom.spec.whatwg.org/, 2025a. Web Hypertext Application Technology Working Group. Accessed: 2025-06-02.

WHATWG. HTML living standard. https://html.spec.whatwg.org/multipage/, 2025b. Web Hypertext Application Technology Working Group. Accessed: 2025-06-02.

Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. Online-Mind2Web Benchmark, 2025a.

Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. An illusion of progress? assessing the current state of web agents, 2025b.

Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-Mark prompting unleashes extraordinary visual grounding in GPT-4V, 2023. URL https://arxiv.org/abs/2310.11441.

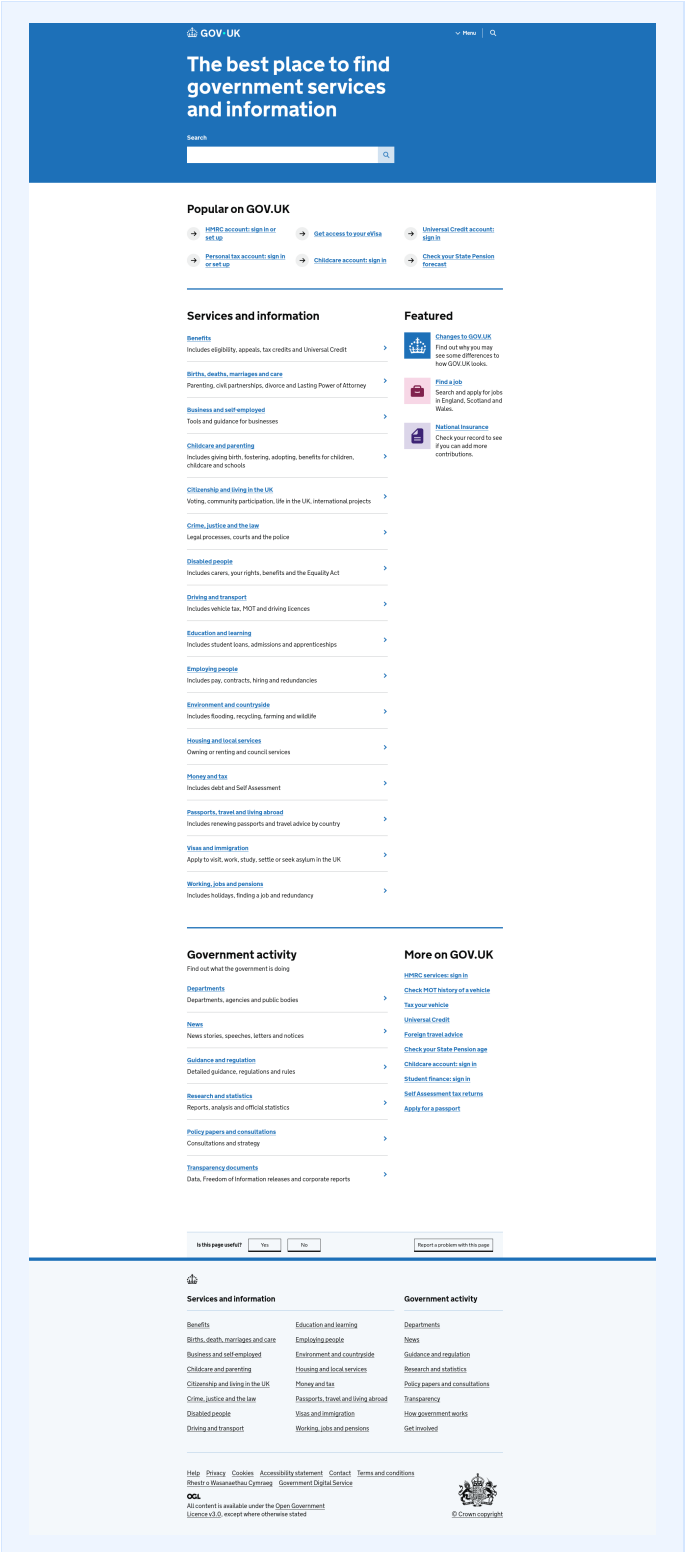
Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded, 2023.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024.

A GUI Snapshot Examples

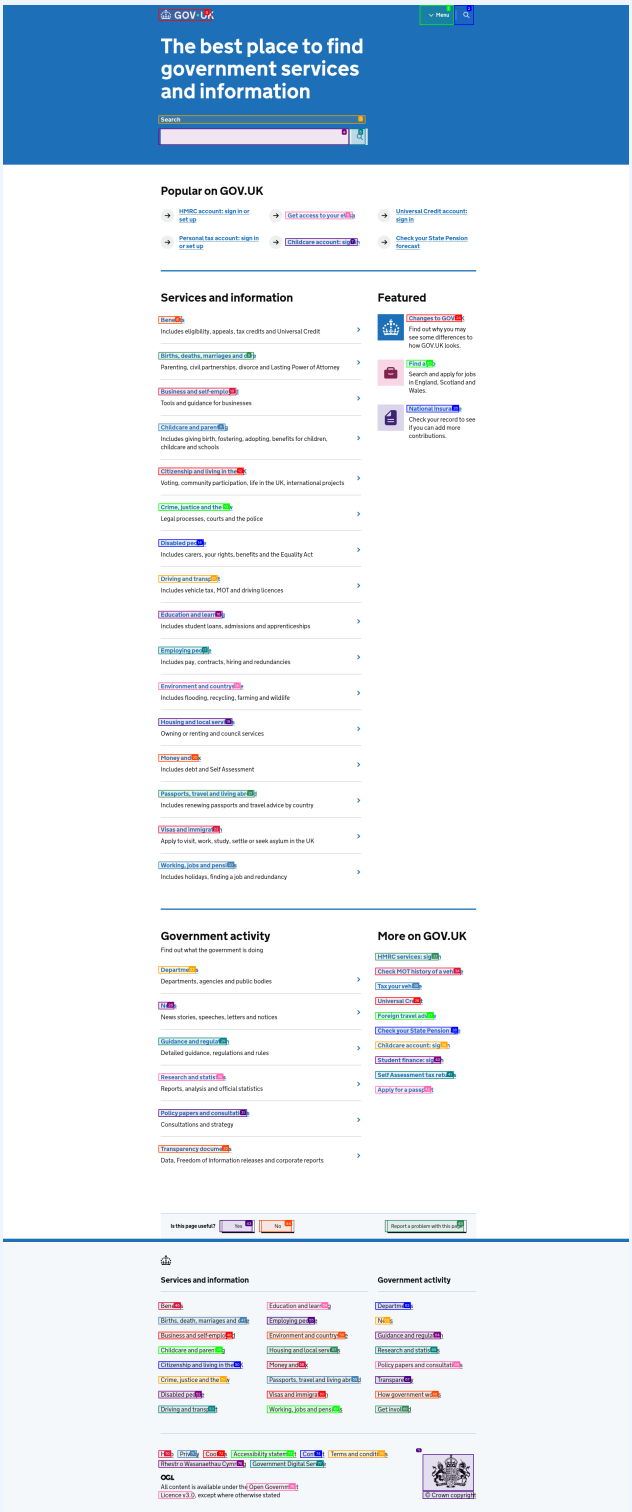
A.1 GUI Snapshot

A GUI snapshot corresponds to a screenshot.



A.2 Grounded GUI Snapshot

A grounded GUI snapshot corresponds to a GUI snapshot (image) enhanced with visual cues – to bridge the lack of LLM vision pixel-precision. Visual cues are coloured bounding boxes identified by numerical indices (Set-of-Mark). The grounding (text) describes actionable elements associated with these identifiers providing (tag) name and text contents.




```
[0] A ""
[1] BUTTON "Menu"
[2] BUTTON "Search GOV.UK"
[3] LABEL "Search"
[4] INPUT ""
[5] BUTTON "Search GOV.UK"
[6] A "Get access to your eVisa"
[7] A "Childcare account: sign in"
[8] A "Benefits"
[9] A "Births, deaths, marriages and care"
[10] A "Business and self-employed"
[11] A "Childcare and parenting"
[12] A "Citizenship and living in the UK"
[13] A "Crime, justice and the law"
[14] A "Disabled people"
[15] A "Driving and transport"
[16] A "Education and learning"
[17] A "Employing people"
[18] A "Environment and countryside"
[19] A "Housing and local services"
[20] A "Money and tax"
[21] A "Passports, travel and living abroad"
[22] A "Visas and immigration"
[23] A "Working, jobs and pensions"
[24] A "Changes to GOV.UK"
[25] A "Find a job"
[26] A "National Insurance"
[27] A "Departments"
[28] A "News"
[29] A "Guidance and regulation"
[30] A "Research and statistics"
[31] A "Policy papers and consultations"
[32] A "Transparency documents"
[33] A "HMRC services: sign in"
[34] A "Check MOT history of a vehicle"
[35] A "Tax your vehicle"
[36] A "Universal Credit"
[37] A "Foreign travel advice"
[38] A "Check your State Pension age"
[39] A "Childcare account: sign in"
[40] A "Student finance: sign in"
[41] A "Self Assessment tax returns"
[42] A "Apply for a passport"
[43] BUTTON "Yes this page is useful"
[44] BUTTON "No this page is not useful"
[45] BUTTON "Report a problem with this page"
[46] A "Benefits"
[47] A "Births, death, marriages and care"
[48] A "Business and self-employed"
[49] A "Childcare and parenting"
[50] A "Citizenship and living in the UK"
[51] A "Crime, justice and the law"
[52] A "Disabled people"
[53] A "Driving and transport"
[54] A "Education and learning"
[55] A "Employing people"
[56] A "Environment and countryside"
[57] A "Housing and local services"
[58] A "Money and tax"
[59] A "Passports, travel and living abroad"
[60] A "Visas and immigration"
[61] A "Working, jobs and pensions"
[62] A "Departments"
[63] A "News"
[64] A "Guidance and regulation"
[65] A "Research and statistics"
[66] A "Policy papers and consultations"
[67] A "Transparency"
[68] A "How government works"
[69] A "Get involved"
[70] A "Help"
[71] A "Privacy"
[72] A "Cookies"
[73] A "Accessibility statement"
[74] A "Contact"
[75] A "Terms and conditions"
[76] A "Rhestr o Wasanaethau Cymraeg"
[77] A "Government Digital Service"
[78] A "Open Government Licence v3.0"
[79] A "$@$ Crown copyright"
```

B Example DOM Snapshots

pizza.html

```
<section class="container" tabindex="3" required="true" type="example">
  <div class="mx-auto" data-topic="products" required="false">
    <h1>Our Pizza</h1>
    <div>
      <div class="shadow-lg">
        <h2>Margherita</h2>
        <p>
         A simple classic: mozzarella, tomatoes and basil.
         An everyday choice!
        </p>
        <button type="button">Add</button>
      </div>
      <div class="shadow-lg">
        <h2>Capricciosa</h2>
        <p>
          A rich taste: mozzarella, ham, mushrooms, artichokes and olives.
          A true favourite!
        </p>
        <button type="button">Add</button>
      </div>
    </div>
  </div>
</section>
```

$D2Snap$(pizza.html) | $r_e = r_a = r_t = 0.3$ (Δ_{Raw} 68%)

```
<section class="container" required="true" type="example">
  <div class="mx-auto" required="false">
    # Our Pizza
    <div>
      ## Margherita
      A simple classic: mozzarella, tomatoes and basil.
      An everyday choice!
      <button type="button">Add</button>
      ## Capricciosa
      A rich taste: mozzarella, ham, mushrooms, artichokes and olives.
      A true favourite!
      <button type="button">Add</button>
    </div>
  </div>
</section>
```

$D2Snap$(pizza.html) | $r_e = 0.9, r_a = 0.3, r_t = 0.6$ (Δ_{Raw} 46%)

```
<section class="container" required="true" type="example">
  # Our Pizza
  ## Margherita
  A simple classic: mozzarella, tomatoes and basil.
  <button type="button">Add</button>
  ## Capricciosa
  A rich taste: mozzarella, ham, mushrooms, artichokes and olives.
  <button type="button">Add</button>
</section>
```

$D2Snap$(pizza.html) | $r_e = r_a = 1, r_t = 0$ (Δ_{Raw} 35%)

```
# Our Pizza
## Margherita
A simple classic: mozzarella, tomatoes and basil.
An everyday choice!
<button>Add</button>
## Capricciosa
A rich taste: mozzarella, ham, mushrooms, artichokes and olives.
A true favourite!
<button>Add</button>
```

C LLM-Elicited[1] Attribute Scoring

MODEL INPUT

For each standard HTML attribute: Score its UI-features, i.e., its semantics in the inherent user interface ('S'), using a ratio scale on the unit interval $[0, 1]$. Subsequently calibrate these scores against the global frequency of each attribute so that, in a DOM representative of an average document, the fraction of attribute occurrences with a score of at most 's' is approximately s, for every s in $[0, 1]$. Respond with a list of tuples (attribute, S).

MODEL OUTPUT

```
('accept', 0.24)
('accept-charset', 0.01)
('accesskey', 0.00)
('action', 0.27)
('alt', 0.46)
('aria-*', 0.62)
('autocapitalize', 0.01)
('autocomplete', 0.15)
('autofocus', 0.16)
('autoplay', 0.03)
('checked', 0.36)
('cite', 0.01)
('class', 0.77)
('content', 0.05)
('contenteditable', 0.15)
('controls', 0.36)
('coords', 0.00)
('datetime', 0.15)
('disabled', 0.34)
('download', 0.02)
('draggable', 0.01)
('dropzone', 0.00)
('enctype', 0.17)
('for', 0.30)
('form', 0.26)
('hidden', 0.28)
('href', 0.91)
('hreflang', 0.02)
('http-equiv', 0.01)
('id', 0.68)
('ismap', 0.00)
('label', 0.31)
('lang', 0.30)
('loop', 0.04)
('max', 0.23)
('maxlength', 0.21)
('media', 0.03)
('method', 0.27)
('min', 0.23)
('minlength', 0.21)
('multiple', 0.26)
('muted', 0.04)
('name', 0.40)
('novalidate', 0.02)
('pattern', 0.22)
('ping', 0.00)
('placeholder', 0.25)
('poster', 0.03)
('preload', 0.02)
('readonly', 0.33)
('rel', 0.18)
('required', 0.33)
('role', 0.32)
('selected', 0.35)
('shape', 0.00)
('size', 0.04)
('sizes', 0.19)
('spellcheck', 0.02)
('src', 0.83)
('srcset', 0.20)
('step', 0.22)
('style', 0.10)
('tabindex', 0.16)
('target', 0.17)
('title', 0.29)
('type', 0.38)
('usemap', 0.01)
('value', 0.43)
('wrap', 0.01)

('*', 0.00) [ed.][2]
```

[1] GPT-4o (`gpt-4o-2024-11-20` (OpenAI, 2025d))
[2] Manual edit by the author.

D D2Snap

INPUT: ELEMENT_NODE $\equiv$ DOM, r_e, r_a, $r_t \in [0, 1]$, $\mathcal{A}$
OUTPUT: ELEMENT_NODE $\equiv$ DOM
PROCEDURE D2Snap $\in \mathcal{O}(|\backslash\text{text}\{\text{DOM}\}|)$[§]:
  // POST-ORDER DOM TRAVERSAL:
  for CHILD_ELEMENT_NODE **of** $\text{ELEMENT_NODE}_{\text{Children}}$:
    D2Snap(CHILD_ELEMENT_NODE, r_e, r_a, r_t, $\mathcal{A}$)

  // TEXT NODE SUB-PROCEDURE:
  for TEXT_NODE **of** $\text{ELEMENT_NODE}_{\text{Text}}$:
    SENTENCES $\leftarrow$ Tokenize($\text{TEXT_NODE}_{\text{TextContent}}$)
    $\text{SENTENCES}_{\text{ranked}} \leftarrow \text{TextRank}_{\text{Sentence}}(\text{SENTENCES})$
    $\text{SENTENCES}_{\text{selected}} \leftarrow \text{SENTENCES}_{\text{ranked}}[0 : \max(\lceil(1 - r_t) \cdot |\text{SENTENCES}|\rceil, 1)]$
    $\text{SENTENCES}_{\text{re-ordered}} \leftarrow \text{Order}_{\text{Natural}}(\text{SENTENCES}_{\text{selected}})$
    $\text{TEXT_NODE}_{\text{TextContent}} \leftarrow \text{Join}(\text{SENTENCES}_{\text{re-ordered}})$

  // ATTRIBUTE NODE SUB-PROCEDURE:
  for ATTRIBUTE_NODE **of** $\text{ELEMENT_NODE}_{\text{Attributes}}$:
    if $\mathcal{A}(\text{ATTRIBUTE_NODE}_{\text{Name}}) < r_a$:
      Remove(ATTRIBUTE_NODE)

  // ELEMENT NODE SUB-PROCEDURE:
  switch Class(ELEMENT_NODE):
    case Class.ACTIONABLE:
      break
    case Class.TEXT_FORMATTING:
      TEXT_NODE $\leftarrow$ Markdown(ELEMENT_NODE)
      Replace(ELEMENT_NODE, TEXT_NODE)
      break
    default:
      d $\leftarrow \text{Depth}_{\text{frozen}}$(ELEMENT_NODE)
      if d > 1 **and** $\lfloor \text{d} \cdot r_e \rfloor > \lfloor (\text{d} - 1) \cdot r_e \rfloor$:
        Replace(ELEMENT_NODE, $\text{ELEMENT}_{\text{Children}}$)
        break

  // RETURN REFERENCE TO MUTATED DOM:
  return ELEMENT_NODE

[§] We presume that *TextRank*, which is embedded in D2Snap's text sub-procedure, is bounded by sentence count and number of iterations to convergence.

E Annotated Dataset Entry


```
{
  "id": "govuk-0",
  "trajectories": [
    [
      {
        "css_selector": "li.gem-c-cards__list-item:nth-of-type(15) > div.gem-c-cards__list-item-wrapper > h3.gem-c-cards__sub-heading
          .govuk-heading-s > a.govuk-link.gem-c-cards__link.gem-c-force-print-link-styles",
        "bounding_box": [ [ 480, 2473 ], [ 609, 108 ] ],
        "bu_identifier": 22
      }
    ],
    [
      {
        "css_selector": "li.govuk-footer__list-item:nth-of-type(15) > a.govuk-footer__link",
        "bounding_box": [ [ 810, 4222 ], [ 179, 18 ] ],
        "bu_identifier": 60
      }
    ],
    [
      {
        "css_selector": "#search-main-ec8e678d",
        "bounding_box": [
          [ 480, 376 ], [ 581, 50 ] ],
        "bu_identifier": 4
      },
      {
        "css_selector": "div.gem-c-search.govuk-\\!-display-none-print.gem-c-search--large.gem-c-search--homepage
          .gem-c-search--on-govuk-blue.gem-c-search--separate-label.govuk-\\!-margin-bottom-0 > div.gem-c-search__item-wrapper > div
          .gem-c-search__item.gem-c-search__submit-wrapper:nth-of-type(2) > button.gem-c-search__submit",
        "bounding_box": [
          [ 1060, 376 ],
          [ 50, 50 ]
        ],
        "bu_identifier": 5
      }
    ],
    [
      {
        "css_selector": "li.gem-c-layout-super-navigation-header__dropdown-list-item:nth-of-type(15) > a.govuk-link
          .gem-c-layout-super-navigation-header__navigation-second-item-link"
      }
    ],
    [
      {
        "css_selector": "a.gem-c-layout-super-navigation-header__search-item-link"
      },
      {
        "css_selector": "div.gem-c-search.govuk-\\!-display-none-print.gem-c-search--large.gem-c-search--on-white
          .gem-c-search--separate-label.govuk-\\!-margin-bottom-0 > div.gem-c-search__item-wrapper > div
          .gem-c-search__item.gem-c-search__submit-wrapper:nth-of-type(2) > button.gem-c-search__submit"
      }
    ]
  ]
}
```

F Evaluation System Prompt

To control the system prompt used in our evaluation, we define a generic template. The template contains variable tags (**{{ <TAG> }}**) for each snapshot specific part of the prompt. Below are the template, and specific substitutes per snapshot class.

F.1 Template

```
# Identity

You are an AI agent that solves web-based tasks on behalf of a human user.

> Assume that today is July 16, 2025.

# Instructions

The user provides you with a web-based task, and serialised state of the web application to solve the task with.
A task may be iterative, so it may not be possible to solve the task completely, but only partially with the given state.

Based on the state representation, your goal is to suggest all elements required to interact with in order to solve the task.
It is important that the list of elements corresponds to a complete interaction trajectory. High precision when
referencing target elements is key in order to be able to reproduce the interactions on the respective user interface.

## Input

### Task

The web-based task is denoted with the prefix `TASK:`, e.g. "TASK: Show 4-star hotels in Amsterdam".

### Snapshot

{{ SNAPSHOT_DESCRIPTION }}

# Output

Follow these rules when considering an element for interaction:

- In case there are multiple trajectories to solve the task, rank them in memory according to human-readability
  and choose the highest ranked alternative
- If there are alternative elements per trajectory which seem to do the same thing, choose the most expressive alternative
- Suppose there are only point and click actions, so never imply any other interaction
- Try not to over-suggest, i.e., suggest only mandatory elements to solve the task with

## Schema

{{ SCHEMA_DESCRIPTION }}

# Examples

Consider the web-based task "TASK: Calculate the sum of 2 and 3.".

<user_query>
  TASK: Calculate the sum of 2 and 3.
</user_query>

<user_query>
  {{ EXAMPLE_SNAPSHOT }}
</user_query>

<assistant_response>
  {{ EXAMPLE_RESPONSE }}
</assistant_response>
```

F.2 GUI

{{ SNAPSHOT_DESCRIPTION }}

```
You are provided with a screenshot, namely the rendered GUI.
```

{{ SCHEMA_DESCRIPTION }}

```
Target elements by their spatial center pixel coordinates. This means, refer to
them through an x (horizontal) and a y (vertical) pixel coordinate relative to
the origin, which is in the top left corner of the image.
```

{{ EXAMPLE_SNAPSHOT }}[a]

```
``` base64
...
```
```

[a] https://github.com/webfuse-com/D2Snap/blob/main/.github/gui.b64-example.png

{{ EXAMPLE_RESPONSE }}

```
``` json
[
  {
    "elementDescription": "Field that contains the mathematical expression to
      be solved.",
    "x": 100,
    "y": 47
  },
  {
  "elementDescription": "Button that triggers the calculation of the provided
    mathematical expression.",
    "x": 100,
    "y": 197
  }
]
```
```

F.3 Grounded GUI

{{ SNAPSHOT_DESCRIPTION }}

```
You are provided with two means of input:

1. A screenshot of the browser with bounding boxes and related numeric identifiers.
2. A list of interactive elements with format `[index] type "text"`. `index` is the
numeric identifier, `type` is an HTML element type (button, input, etc.), and
`text` is the element description.

> Numeric identifiers across means of input are consistent.
```

{{ SCHEMA_DESCRIPTION }}

```
Target elements by their numeric identifiers as given across both means of input.
```

{{ EXAMPLE_SNAPSHOT }}[a]

```
``` base64
...
```
```

```
```
[0] INPUT "Type expression"
[1] BUTTON "Solve"
[2] A ""
```
```

[a] https://github.com/webfuse-com/D2Snap/blob/main/.github/gui.b64-example.bu.png

{{ EXAMPLE_RESPONSE }}

```
``` json
[
  {
    "elementDescription": "Field that contains the mathematical expression to
      be solved.",
    "identifier": 0
  },
  {
    "elementDescription": "Button that triggers the calculation of the provided
      mathematical expression.",
    "identifier": 1
  }
]
```
```

F.4 DOM/D2Snap

{{ SNAPSHOT_DESCRIPTION }}

```
You are provided with HTML, namely a serialised DOM.
```

{{ SCHEMA_DESCRIPTION }}

```
Target elements by shortest unique CSS selector, e.g. `#app > button:first-child`.
```

{{ EXAMPLE_SNAPSHOT }}

```
``` html
<main>
  <h1>Calculator</h1>
  <input id="expression" class="field" type="text" placeholder="3 * 4">
  <button id="submit" type="button">Solve</button>
  <div id="result">
    <span></span>
  </div>
```
```

{{ EXAMPLE_RESPONSE }}

```
``` json
[
  {
    "elementDescription": "Field that contains the mathematical expression to
      be solved.",
    "cssSelector": "#expression"
  },
  {
   "elementDescription": "Button that triggers the calculation of the provided
     mathematical expression.",
    "cssSelector": "#submit"
  }
]
```
```